Artificial Intelligence (CSA1711) — Assessment Tool 3: Case Studies — Model Answers

Assessment Tool 3— Case Studies (CO3)

Case Study 1 — Smart Medical Diagnosis System

Knowledge base: F = Fever, C = Cough, R = Rash, B = Breathlessness, Fl = Possible Flu, Me = Possible Measles, Pn = Possible Pneumonia

- Rule I: $F \land C \rightarrow Fl$
- Rule II: $F \land R \rightarrow Me$
- Rule III: $C \land B \rightarrow Pn$
- Facts for Patient A: F = True, C = True, B = True (R is not asserted, so it is treated as not established)

(a) Propositional representation

Symbol	Meaning
F	Patient has Fever
C	Patient has Cough
R	Patient has Rash
B	Patient has Breathlessness
Fl	Possible Flu
Me	Possible Measles
Pn	Possible Pneumonia

Rules: **R1: $F \land C \rightarrow Fl$**, **R2: $F \land R \rightarrow Me$**, **R3: $C \land B \rightarrow Pn$** Facts: **F = True, C = True, B = True**

(b) Forward Chaining for Patient A

Step	Facts available	Rule checked	Fires?	Fact derived
1	F, C, B	R1: $F \land C \rightarrow Fl$	Yes (F, C both True)	Fl (Possible Flu)
2	F, C, B	R2: $F \land R \rightarrow Me$	No (R not established)	—
3	F, C, B	R3: $C \land B \rightarrow Pn$	Yes (C, B both True)	Pn (Possible Pneumonia)

Result: Patient A is diagnosed with **Possible Flu** and **Possible Pneumonia.** Measles cannot be derived because Rash was never asserted.

(c) Backward Chaining — Goal: "Patient A has Pneumonia"

Goal: **Pn** Only Rule III concludes Pn, so: $Pn \leftarrow C \land B$

Goal reduction tree:

```
Goal: Pn
 └── Rule III: C ∧ B → Pn
      ├── Sub-goal: C  → matches fact (C = True)  ✓ satisfied
      └── Sub-goal: B  → matches fact (B = True)  ✓ satisfied
 => Both sub-goals satisfied → Pn is PROVED TRUE
```

Conclusion: Both sub-goals reduce directly to known facts, so the goal "Patient A has Pneumonia" is confirmed.

Case Study 2 — Autonomous Traffic Management Agent

Rules: - Rule 1: ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) - Rule 2: ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) - Rule 3: ∀x∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)

Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Unification with Rule 1

Rule 1 pattern: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)

- Unify **Vehicle(x)** with **Vehicle(Ambulance)** → θ_1 = {x / Ambulance}
- Apply θ_1 to the second literal: **EmergencyType(x)** becomes **EmergencyType(Ambulance)**, which matches the fact EmergencyType(Ambulance) directly — no further binding required.

MGU: θ = {x / Ambulance}, giving the derived fact **ClearPath(Ambulance)**.

(b) Resolution proof of Proceed(CarA)

Step 1 — Convert to CNF (∀-quantifiers dropped; variables implicitly universal):

- C1: ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x) — (Rule 1)
- C2: ¬ClearPath(x) ∨ GreenSignal(x) — (Rule 2, GreenSignal conjunct)
- C3: ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x) — (Rule 3)
- C4: Vehicle(Ambulance)
- C5: EmergencyType(Ambulance)
- C6: Vehicle(CarA)
- C7: Behind(CarA, Ambulance)
- C8: ¬Proceed(CarA) — (negated goal)

Step 2 — Resolution:

1. Resolve C1 with C4 {x/Ambulance} → ¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)

2. Resolve (1) with C5 → **ClearPath(Ambulance)**
3. Resolve C2 with (2) {x/Ambulance} → **GreenSignal(Ambulance)**
4. Resolve C3 with C8 {x/CarA} → ¬Vehicle(CarA) ∨ ¬Behind(CarA,y) ∨ ¬GreenSignal(y)
5. Resolve (4) with C6 → ¬Behind(CarA,y) ∨ ¬GreenSignal(y)
6. Resolve (5) with C7 {y/Ambulance} → ¬GreenSignal(Ambulance)
7. Resolve (6) with (3) → **Empty clause □**

Deriving □ proves, by refutation, that **Proceed(CarA) is TRUE**.

(c) Sufficiency for two simultaneous emergency vehicles

The KB is **not sufficient**. Gaps:

- No rule orders or prioritises two ambulances that both satisfy Vehicle(x) ∧ EmergencyType(x) at the same time — both would independently derive ClearPath and GreenSignal, with no mechanism to sequence them.
- If the two emergency vehicles approach from conflicting directions, Rule 2 could assign GreenSignal to both simultaneously, since nothing in the KB expresses that two directions are mutually exclusive.

Additional rules/facts needed: - A priority predicate, e.g. `∀x∀y: EmergencyType(x) ∧ EmergencyType(y) ∧ CloserToIntersection(x,y) → Priority(x) ∧ ¬ClearPath(y)` (holds ClearPath for the second vehicle until the first has passed). - A conflict/exclusion rule: `ConflictingDirection(x,y) → ¬(GreenSignal(x) ∧ GreenSignal(y))`. - Facts recording arrival order/distance-to-intersection for each emergency vehicle so priority can be computed.

Without these, the KB can entail an unsafe state (two conflicting green signals at once), so it must be extended before it can be trusted with concurrent emergencies.

Case Study 3 — Agricultural Expert Reasoning System

- P1: SoilDry → IrrigationNeeded
- P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
- P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
- Facts: SoilDry = True, CropWheat = True

(a) Conversion to CNF

P1: SoilDry → IrrigationNeeded Eliminate implication: ¬SoilDry ∨ IrrigationNeeded — *(already CNF)*

P2: (IrrigationNeeded ∧ CropWheat) → ApplyDripMethod Eliminate implication: ¬(IrrigationNeeded ∧ CropWheat) ∨ ApplyDripMethod Move ¬ inward (De Morgan): ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod — *(already CNF)*

P3: (¬ApplyDripMethod ∧ CropWheat) → CropAtRisk Eliminate implication: ¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk Move ¬ inward (De Morgan): (ApplyDripMethod ∨ ¬CropWheat) ∨ CropAtRisk Simplify: ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk — *(already CNF)*

Facts as unit clauses: SoilDry, CropWheat

(b) Resolution Refutation — prove ApplyDripMethod

Clauses: C1: ¬SoilDry ∨ IrrigationNeeded · C2: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod · C3: SoilDry · C4: CropWheat · C5 (negated goal): ¬ApplyDripMethod

```
1. C1 + C3  --(resolve on SoilDry)-->  IrrigationNeeded
(C6)
2. C6 + C2  --(resolve on IrrigationNeeded)-->  ¬CropWheat ∨ ApplyDripMethod
(C7)
3. C7 + C4  --(resolve on CropWheat)-->  ApplyDripMethod
(C8)
4. C8 + C5  --(resolve on ApplyDripMethod)-->  □  (empty clause)
```

The empty clause is derived, so **ApplyDripMethod is proved true**.

(c) If the fact "SoilDry" is removed

- C1 (¬SoilDry ∨ IrrigationNeeded) can no longer resolve with anything, because the unit clause SoilDry is gone. **IrrigationNeeded can no longer be derived.**
- Since IrrigationNeeded is not derivable, P2's antecedent is not satisfied, so **ApplyDripMethod can no longer be derived** either — in strict (open-world/monotonic) entailment it is simply *unproven*, neither true nor false.
- Whether **CropAtRisk** follows now depends on the reasoning convention used:
  - **Under strict monotonic FOL/propositional entailment:** ApplyDripMethod is unproven, so ¬ApplyDripMethod cannot be asserted either, and CropAtRisk **cannot be proved**.
  - **Under the Closed-World Assumption (typical of rule-based expert systems):** anything not provable is treated as false, so ApplyDripMethod = False ⇒ ¬ApplyDripMethod = True. Combined with CropWheat = True, P3's antecedent is satisfied, so **CropAtRisk = True is inferred**.
- This illustrates that removing a supporting fact breaks the original derivation chain (SoilDry → IrrigationNeeded → ApplyDripMethod), and the final conclusion becomes convention-dependent rather than automatic — a key distinction between strict logical entailment and default/non-monotonic reasoning used in practical expert systems.

Case Study 4 — Wumpus World Knowledge Agent

Percepts: Stench[1,2], Breeze[1,1], Glitter[2,2]

Rules: 1) Stench[1,2] → WumpusAdjacent[1,2] 2) Breeze[1,1] → PitAdjacent[1,1] 3) Glitter[x,y] → GoldAt[x,y] 4) ¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Belief state via Modus Ponens

Percept (fact)	Rule applied	Modus Ponens result
Stench[1,2] = True	Rule 1	WumpusAdjacent[1,2] = True
Breeze[1,1] = True	Rule 2	PitAdjacent[1,1] = True
Glitter[2,2] = True	Rule 3	GoldAt[2,2] = True

Derived conclusions: WumpusAdjacent[1,2], PitAdjacent[1,1], GoldAt[2,2].

(b) Forward Chaining — locating Wumpus and Pit

1. WumpusAdjacent[1,2] ⇒ Wumpus ∈ {[1,1], [1,3], [2,2]} (the cells adjacent to [1,2]).
2. The agent started at [1,1] and perceived no harm there, so by Rule 4 [1,1] is already known Safe ⇒ Wumpus ≠ [1,1]. This eliminates [1,1] ⇒ **Wumpus ∈ {[1,3], [2,2]}**.
3. PitAdjacent[1,1] ⇒ Pit ∈ {[1,2], [2,1]} (the cells adjacent to [1,1]).
4. No further percepts (e.g., breeze at [2,1] or stench at [2,2]) are available yet, so these remain the tightest sets derivable: **Wumpus ∈ {[1,3],[2,2]}**, **Pit ∈ {[1,2],[2,1]}**.

(c) Is the path [1,1] → [1,2] → [2,2] safe?

- [1,2] is a candidate Pit location (from step (b), Pit ∈ {[1,2],[2,1]}). Since ¬Pit[1,2] cannot be established, Rule 4's condition ¬Wumpus[1,2] ∧ ¬Pit[1,2] is **not satisfiable**, so Safe[1,2] cannot be derived.
- Therefore the second step of the path, entering [1,2], is **not certified safe** — the agent cannot rule out a pit there with the current knowledge.

Conclusion: The path **[1,1] → [1,2] → [2,2] is NOT safe** to take as-is. The agent should first gather more evidence (e.g., check for breeze at [2,1]) to eliminate [1,2] as the pit location before proceeding; until then, logical inference cannot certify Safe[1,2].